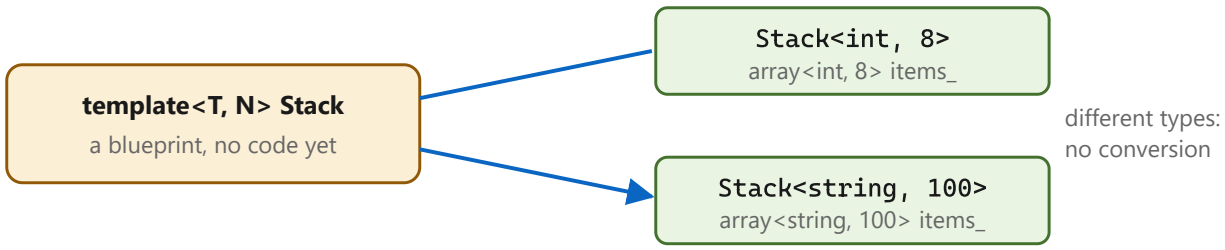
Class Templates

The same idea for classes: one blueprint, one class per set of arguments. Non-type parameters, class template argument deduction, full and partial specialization, alias and variable templates. Assumes Function Templates.

Class templates

Same idea for classes: `vector<T>`, `map<K,V>`, `unique_ptr<T>`.



Each combination of arguments is its own class; members are generated only when used.

```
// a type parameter, a NON-TYPE parameter, with a default
template <typename T, std::size_t Capacity = 8>
class Stack {
    std::array<T, Capacity> items_;
    std::size_t size_ = 0;
public:
    bool push(const T& v) {
        if (size_ == Capacity) return false;
        items_[size_++] = v;
        return true;
    }
    T pop() { return items_[--size_]; }
    bool empty() const { return size_ == 0; }
};

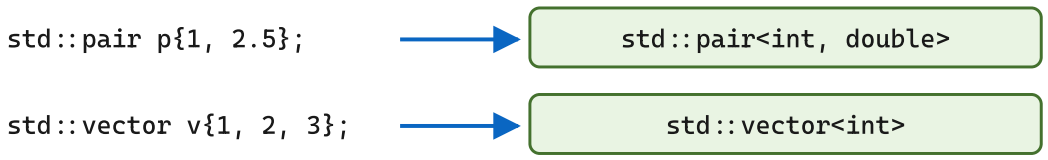
Stack<int> a;                // Stack<int, 8>
Stack<std::string, 100> b;   // a different, unrelated type

// C++17 CTAD: class template argument deduction → std::pair<int, double>
std::pair p{1, 2.5};
std::vector v{1, 2, 3};      // std::vector<int>
```

- **Non-type parameters** are compile-time constants (how `std::array<int,5>` knows its size).
- Each instantiation is a separate, unrelated class. Members are instantiated only if called.

> **WHY TEMPLATES LIVE IN HEADERS** The compiler needs the body at instantiation. In a `.cpp`, others see only a declaration: linker errors. Use headers, or explicit instantiation (`template class Stack<int>;`).

Class template argument deduction (C++17)



When constructors are not enough, write a deduction guide: template <class T> Box(T) -> Box<T>;

Class template argument deduction (C++17) removes the angle brackets you can infer.

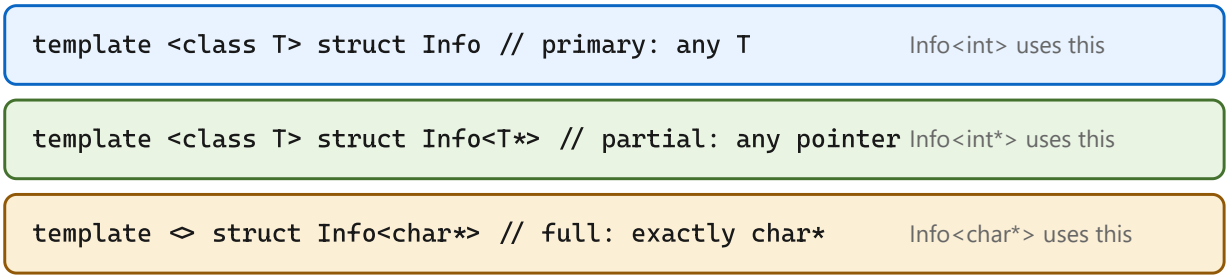
```cpp
std::pair p{1, 2.5};                    // std::pair<int, double>
std::vector v{1, 2, 3};                 // std::vector<int>
std::array a{1, 2, 3};                  // std::array<int, 3>

template <typename T> struct Box { T value; };
// a user-defined deduction guide (needed for aggregates before C++20)
template <typename T> Box(T) -> Box<T>;
Box b{std::string("x")};                // Box<std::string>
```

The compiler deduces the class's arguments from the constructor call. A **deduction guide** tells it how when the constructors alone are not enough. All of the above was compiled and checked.

Specialization

A **specialization** replaces the generic template for specific arguments.



Functions cannot be partially specialized; overload them instead.

```cpp
template <typename T>
struct Printer {                                // primary template
    static void print(const T& v) { std::cout << v << '\n'; }
};

template <>                                     // full specialization for bool
struct Printer<bool> {
    static void print(bool v) { std::cout << (v ? "yes" : "no") << '\n'; }
};

template <typename T>
// PARTIAL specialization: any pointer type
struct Printer<T*> {
    static void print(T* p) { std::cout << "ptr to " << *p << '\n'; }
};
```

KIND	MEANING	ALLOWED FOR
Full specialization	all parameters fixed: `template<> ... Printer<bool>`	class and function templates
Partial specialization	a pattern: `Printer<T*>`	**class templates only**

FUNCTIONS: PREFER OVERLOADS No partial specialization for function templates.

Overload instead; an exact non-template match wins. Function specializations do not take part in overload resolution.

Alias templates, variable templates and more

TOOL	EXAMPLE	USE
Alias template	`template <typename T> using Table = std::map<std::string, std::vector<T>>;`	a shorter name that still takes a parameter; unlike `typedef` it can be templated
Variable template (C++14)	`template <typename T> constexpr T zero = T{};`	a constant per type: `zero<int>`, `zero<double>`. The standard's `std::is_integral_v<T>` is one
Template template parameter	`template <template <typename ...> class C, typename T>`	pass a *template* (`std::vector`) rather than a type: `makeTwo<std::vector>(3, 4)`
Non-type parameter	`template <typename T, std::size_t N>`	a value as parameter; `auto` accepts any type of value in C++17: `template <auto V>`
Abbreviated template (C++20)	`void f(auto x)`	each `auto` parameter is an implicit template parameter

```cpp
template <typename T> using Table = std::map<std::string, std::vector<T>>;
// std::map<std::string, std::vector<int>>
Table<int> t;

// zero<int> is 0, zero<double> is 0.0
template <typename T> constexpr T zero = T{};

template <template <typename ...> class Container, typename T>
Container<T> makeTwo(T a, T b) { Container<T> c; c.push_back(a);
c.push_back(b); return c; }
// a vector<int> with two elements
auto v = makeTwo<std::vector>(3, 4);
```

Unpacking with `std::index_sequence`

To call a function with the elements of a `std::tuple` you need the indices `0, 1, 2 ...` as a pack. `std::make_index_sequence<N>` generates it:

```cpp
template <typename F, typename Tuple, std::size_t ... I>
// expands to get<0>, get<1>, ...
auto applyImpl(F f, const Tuple& t, std::index_sequence<I ...>) { return
f(std::get<I>(t) ...); }

template <typename F, typename Tuple>
auto applyTuple(F f, const Tuple& t) { return applyImpl(f, t,
std::make_index_sequence<std::tuple_size_v<Tuple>>{}); }

// 6 (compiled)
applyTuple([](int x, int y, int z) { return x + y + z; }, std::tuple{1, 2,
3});
```

C++17 ships this as `std::apply`. Know the idea: a pack of integers lets you expand an expression once per element.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Class templates	C++ Primer	16.1 Defining a template	681 to 706
	Programming: Principles and Practice	18.1 to 18.3 Templates, generalizing Vector, range checking and exceptions	527 to 541
Specialization	C++ Primer	16.5 Template specializations	735 to 743